

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

Отчёт
по лабораторной работе №8
по курсу “Логика и основы алгоритмизации в инженерных задачах”
на тему “Обход графа в ширину”

Выполнили:

студенты группы 24ВВВ4:

Симаков А.С.

Орлов Р.А.

Принял:

к. т. н., доцент Юрова О.В.

к. э. н., доцент Акифьев И.В.

Пенза 2025

Цель работы:

Разработать программу, где реализуется поиск того или иного значения с клавиатуры

Лабораторное задание:

Задание 1

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа G. Выведите матрицу на экран.
2. Для сгенерированного графа осуществите процедуру обхода в ширину, реализованную в соответствии с приведенным выше описанием. При реализации алгоритма в качестве очереди используйте класс queue из стандартной библиотеки C++.
3. Реализуйте процедуру обхода в ширину для графа, представленного списками смежности.

Результат работы программы:

```
Введите количество вершин графа: 7
Матрица смежности (7x7):
 0 1 2 3 4 5 6
0 0 1 1 1 1 0 0
1 1 0 1 0 0 0 1
2 1 1 0 1 1 0 0
3 1 0 1 0 1 0 0
4 1 0 1 1 0 0 0
5 0 0 0 0 0 0 1
6 0 1 0 0 0 1 0

Списки смежности:
Вершина 0: 4 3 2 1
Вершина 1: 6 2 0
Вершина 2: 4 3 1 0
Вершина 3: 4 2 0
Вершина 4: 3 2 0
Вершина 5: 6
Вершина 6: 5 1

Введите начальную вершину для обхода (0-6): 6
Обход в ширину (матрица смежности), начиная с вершины 6: 6 1 5 0 2 3 4
Обход в ширину (списки смежности), начиная с вершины 6: 6 5 1 2 0 4 3
```

Листинг:

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
#include <time.h>
#include <queue>

using namespace std;

void DFS(int** G, int numG, int*visited, int s){
    visited[s] = 1;
    printf("%3d",s);
```

```

        for (int i = 0; i < numG; i++){
            if (G[s][i] == 1 && visited[i] == 0)
                DFS(G, numG, visited, i);
        }

    }

void BFS(int** G, int numG, int*visited, int s) {
    queue<int> q;
    int v;

    visited[s]=1;
    q.push(s);

    while (!q.empty()) {
        v = q.front();
        q.pop();
        printf("%3d", v);

        for (int i = 0; i < numG; i++) {
            if (G[v][i] == 1 && visited[i] == 0) {
                q.push(i);
                visited[i] = 1;
            }
        }
    }
}

int main() {

    int** G;
    int numG, current;
    int* visited;

    printf("Input number of vertission: ");
    scanf("%d", &numG);

    visited = (int*)malloc(numG * sizeof(int));
    G = (int**)malloc(numG * sizeof(int*));
    for (int i = 0; i < numG; i++){
        G[i] = (int*)malloc(numG * sizeof(int));
    }

    for (int i = 0; i < numG; i++){
        visited[i] = 0;
        for (int j = i; j < numG; j++){
            G[i][j] = G[j][i] = (i==j ? 0 : rand() % 2);
        }
    }

    for (int i = 0; i < numG; i++){
        for (int j= 0; j < numG; j++){
            printf("%3d", G[i][j]);
        }
        printf("\n");
    }
}

```

```

        printf("Input start vert: ");
        scanf("%d", &current);

        printf("\nPath: ");
        BFS(G, numG, visited, current);

    for (int i = 0; i < numG; i++)
        free(G[i]);
    free(G);
    free(visited);

    _getch();

    return 0;
}

```

Выводы:

В ходе выполнения лабораторной работы были разработаны программы, выполняющие указанные в лабораторной работе задачи. Результаты работ программ совпали с результатами трассировок, следовательно, программы работают без ошибок.

Получили опыт в создании проектов в среде MicrosoftVisualStudio.